

GitHub Actions CI/CD Tutorial: Automate Build → Test → Push → Deploy

A hands-on guide to automating your Docker workflow with GitHub Actions — from your first pipeline to full production deployments.

Table of Contents

1. [What is CI/CD?](#)
 2. [GitHub Actions Core Concepts](#)
 3. [Your First Workflow](#)
 4. [CI Pipeline — Build & Test](#)
 5. [Push to Docker Hub](#)
 6. [Deploy to a Server](#)
 7. [Full Pipeline: Flask + Docker + PostgreSQL](#)
 8. [Secrets & Environment Variables](#)
 9. [Advanced Patterns](#)
 10. [Troubleshooting](#)
 11. [Cheat Sheet](#)
-

1. What is CI/CD?

CI (Continuous Integration) — automatically build and test your code every time you push.

CD (Continuous Delivery/Deployment) — automatically push your tested image to a registry and deploy it to a server.

You push code to GitHub



GitHub Actions

```
1. BUILD      2. TEST      3. PUSH
docker build  pytest /      docker push
your image    npm test      → Docker Hub

4. DEPLOY
SSH into server → docker compose pull → up -d
```



Your app is live with the new version 🚀

Why bother?

Without CI/CD

Manually build & test before every deploy

"I forgot to run tests" → broken production

Inconsistent deployment steps

Deploy takes 20 minutes of manual work

With CI/CD

Happens automatically on every push

Tests must pass before deploy

Same steps every time

One `git push` does everything

2. GitHub Actions Core Concepts

File Location

All workflows live in your repo at:

```
your-repo/
├── .github/
│   └── workflows/
│       ├── ci.yml          ← CI pipeline
│       └── deploy.yml      ← deploy pipeline
```

Anatomy of a Workflow

```
yaml
name: My Workflow          # Name shown in GitHub UI

on:                        # TRIGGER: when does this run?
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:                      # Groups of steps
  build:                   # Job name (you choose)
    runs-on: ubuntu-latest # Runner OS

    steps:                 # Individual tasks
      - name: Checkout code
        uses: actions/checkout@v4    # Pre-built action

      - name: Run a command
        run: echo "Hello World"      # Shell command
```

Key Terms

Term	What it is
Workflow	The whole <code>.yaml</code> file — one automated process
Trigger (<code>on</code>)	What starts the workflow (push, PR, schedule, manual)
Job	A group of steps that run on the same machine
Step	A single task — either a <code>run</code> command or a <code>uses</code> action
Action	A reusable plugin (from GitHub Marketplace)
Runner	The virtual machine that runs your job
Secret	Encrypted variable stored in GitHub settings

3. Your First Workflow

Create `.github/workflows/hello.yml`:

```
yaml

name: Hello World

on:
  push:
    branches: [main]

jobs:
  greet:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Say hello
        run: echo "Hello from GitHub Actions!"

      - name: Show repo files
        run: ls -la

      - name: Show current date
        run: date
```

Push this to GitHub and go to the **Actions** tab to watch it run.

4. CI Pipeline — Build & Test

Python Flask Example

```
yaml
```

```
# .github/workflows/ci.yml
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: |
          pip install -r requirements.txt
          pip install pytest

      - name: Run tests
        run: pytest tests/ -v

  build:
    runs-on: ubuntu-latest
    needs: test          # Only runs if 'test' job passes

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Build Docker image
        run: docker build -t my-flask-app:${{ github.sha }} .
```

Node.js Example

```
yaml
name: CI

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Set up Node
        uses: actions/setup-node@v4
        with:
          node-version: "20"
          cache: "npm"           # Cache node_modules between runs

      - name: Install dependencies
        run: npm ci

      - name: Run tests
        run: npm test

      - name: Build Docker image
        run: docker build -t my-node-app:latest .
```

Running Tests with a Database (Service Containers)

Need PostgreSQL during tests? GitHub Actions can spin up service containers:

```
yaml
```

```
jobs:
  test:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_USER: test_user
          POSTGRES_PASSWORD: test_pass
          POSTGRES_DB: test_db
        ports:
          - 5432:5432
        options: >-
          --health-cmd pg_isready
          --health-interval 5s
          --health-timeout 5s
          --health-retries 5

    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: pip install -r requirements.txt pytest

      - name: Run tests
        env:
          DATABASE_URL: postgresql://test_user:test_pass@localhost:5432/test_db
        run: pytest tests/ -v
```

5. Push to Docker Hub

Step 1 — Add Secrets to GitHub

Go to your repo → **Settings** → **Secrets and variables** → **Actions** → **New repository secret**

Add these two secrets:

- `DOCKERHUB_USERNAME` — your Docker Hub username
- `DOCKERHUB_TOKEN` — a Docker Hub access token (create at hub.docker.com → Account Settings → Security)

Step 2 — Build and Push Workflow

yaml


```

# .github/workflows/docker-publish.yml
name: Build and Push to Docker Hub

on:
  push:
    branches: [main]
  release:
    types: [published]          # Also trigger on GitHub releases

jobs:
  build-and-push:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Log in to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKERHUB_USERNAME }
          password: ${ secrets.DOCKERHUB_TOKEN }

      - name: Extract metadata (tags and labels)
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${ secrets.DOCKERHUB_USERNAME }/my-flask-app
          tags: |
            type=sha                                # e.g. sha-abc1234
            type=ref,event=branch                    # e.g. main
            type=semver,pattern={{version}}          # e.g. 1.2.3 (from git tags)
            type=raw,value=latest,enable=${ github.ref == 'refs/heads/main' }

      - name: Build and push
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: ${ steps.meta.outputs.tags }
          labels: ${ steps.meta.outputs.labels }

```

After this runs, your image is live at:

```
docker pull yourusername/my-flask-app:latest
docker pull yourusername/my-flask-app:sha-abc1234
```

Pushing to GitHub Container Registry (GHCR) instead

GHCR is free and built into GitHub — no separate account needed:

yaml

```
- name: Log in to GHCR
  uses: docker/login-action@v3
  with:
    registry: ghcr.io
    username: ${{ github.actor }}
    password: ${{ secrets.GITHUB_TOKEN }} # Auto-provided, no setup needed

- name: Build and push to GHCR
  uses: docker/build-push-action@v5
  with:
    context: .
    push: true
    tags: ghcr.io/${{ github.repository }}:latest
```

6. Deploy to a Server

Method 1 — SSH into your server and pull the new image

Add these secrets to GitHub:

- `SSH_HOST` — your server IP or domain
- `SSH_USER` — SSH username (e.g. `ubuntu`)
- `SSH_PRIVATE_KEY` — your private SSH key content

yaml

```

- name: Deploy to server
  uses: appleboy/ssh-action@v1
  with:
    host: ${ secrets.SSH_HOST }
    username: ${ secrets.SSH_USER }
    key: ${ secrets.SSH_PRIVATE_KEY }
    script: |
      cd /home/ubuntu/my-app
      docker compose pull
      docker compose up -d --remove-orphans
      docker image prune -f

```

Method 2 — Deploy to a VPS with docker-compose

Your server needs:

1. Docker and Docker Compose installed
2. A `docker-compose.yml` referencing your image from Docker Hub/GHCR
3. Your `.env` file with secrets already on the server

Server's `docker-compose.yml` (uses pre-built image, not `build: .`):

```

yaml
services:
  api:
    image: yourusername/my-flask-app:latest    # ← pull from registry
    ports:
      - "5000:5000"
    env_file:
      - .env
    restart: unless-stopped

  db:
    image: postgres:16
    volumes:
      - db-data:/var/lib/postgresql/data
    env_file:
      - .env

volumes:
  db-data:

```

Method 3 — Deploy to Railway / Render / Fly.io (easier)

These platforms watch your GitHub repo and redeploy automatically. No SSH needed.

Fly.io example:

```
yaml
- name: Deploy to Fly.io
  uses: superfly/flyctl-actions/setup-flyctl@master

- run: flyctl deploy --remote-only
  env:
    FLY_API_TOKEN: ${ secrets.FLY_API_TOKEN }
```

7. Full Pipeline: Flask + Docker + PostgreSQL

This is the complete CI/CD pipeline combining everything above:

```
yaml
```

```
# .github/workflows/pipeline.yml
name: CI/CD Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

env:
  IMAGE_NAME: ${ secrets.DOCKERHUB_USERNAME }}/todo-api

jobs:
  # — JOB 1: Test —————
  test:
    name: Run Tests
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_USER: test_user
          POSTGRES_PASSWORD: test_pass
          POSTGRES_DB: test_todos
        ports:
          - 5432:5432
        options: >-
          --health-cmd pg_isready
          --health-interval 5s
          --health-retries 5

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: pip install -r requirements.txt pytest pytest-cov

      - name: Run tests with coverage
```

```
env:
  DATABASE_URL: postgresql://test_user:test_pass@localhost:5432/test_to
run: pytest tests/ -v --cov=app --cov-report=term-missing
```

```
# — JOB 2: Build & Push —————
```

```
build-push:
```

```
name: Build and Push Image
runs-on: ubuntu-latest
needs: test # Don't build if tests fail
if: github.ref == 'refs/heads/main' # Only on main branch
```

```
steps:
```

- uses: actions/checkout@v4
- name: Log in to Docker Hub
uses: docker/login-action@v3
with:
 username: \${ secrets.DOCKERHUB_USERNAME }
 password: \${ secrets.DOCKERHUB_TOKEN }
- name: Build and push
uses: docker/build-push-action@v5
with:
 context: .
 push: true
 tags: |
 \${ env.IMAGE_NAME }:latest
 \${ env.IMAGE_NAME }:\${ github.sha }

```
# — JOB 3: Deploy —————
```

```
deploy:
```

```
name: Deploy to Server
runs-on: ubuntu-latest
needs: build-push # Only deploy after push succeeds
if: github.ref == 'refs/heads/main'
```

```
steps:
```

- name: Deploy via SSH
uses: appleboy/ssh-action@v1
with:
 host: \${ secrets.SSH_HOST }
 username: \${ secrets.SSH_USER }
 key: \${ secrets.SSH_PRIVATE_KEY }
 script: |

```
cd /home/ubuntu/todo-app
```

```
# Pull latest image  
docker compose pull api
```

```
# Restart with zero downtime  
docker compose up -d --no-deps api
```

```
# Cleanup old images  
docker image prune -f
```

```
echo "✅ Deployed ${github.sha} successfully"
```

Job dependency flow:

```
push to main  
  |  
  ▼  
[test] ——— fails? → pipeline stops, no deploy  
  |  
  | passes  
  ▼  
[build-push] ——— builds image, pushes to Docker Hub  
  |  
  ▼  
[deploy] ——— SSHs into server, pulls new image, restarts
```

8. Secrets & Environment Variables

Types of Variables

```
yaml
```

```
# 1. Hardcoded (fine for non-sensitive config)
env:
  NODE_ENV: production
  PORT: "3000"

# 2. GitHub Secrets (for passwords, tokens, keys)
env:
  DB_PASSWORD: ${ secrets.DB_PASSWORD }

# 3. From github context (auto-provided)
env:
  COMMIT_SHA: ${ github.sha }
  BRANCH: ${ github.ref_name }
  REPO: ${ github.repository }
  ACTOR: ${ github.actor }
```

Where to Add Secrets

GitHub Repo → Settings → Secrets and variables → Actions

Repository secrets: available to all workflows in this repo

Environment secrets: scoped to specific environments (staging/production)

Creating Environments (staging vs production)

```
yaml

jobs:
  deploy-staging:
    environment: staging          # Uses 'staging' environment secrets
    steps:
      - run: echo "Deploying to staging"

  deploy-production:
    environment: production      # Uses 'production' environment secrets
    needs: deploy-staging
    steps:
      - run: echo "Deploying to production"
```

With environments you can also require **manual approval** before deploying to production — set this in Settings → Environments → Required reviewers.

9. Advanced Patterns

Cache Docker Layers (faster builds)

yaml

- **name**: Set up Docker Buildx
uses: docker/setup-buildx-action@v3
- **name**: Build and push with cache
uses: docker/build-push-action@v5
with:
 - context**: .
 - push**: **true**
 - tags**: myimage:latest
 - cache-from**: type=gha # Use GitHub Actions cache
 - cache-to**: type=gha,mode=max

Run Only on Changed Files

yaml

```
on:
  push:
    paths:
      - "app/**" # Only trigger if app/ changes
      - "Dockerfile"
      - "requirements.txt"
      - "!docs/**" # Ignore docs changes
```

Matrix Builds (test multiple versions)

yaml

```

jobs:
  test:
    strategy:
      matrix:
        python-version: ["3.10", "3.11", "3.12"]
        os: [ubuntu-latest, windows-latest]

    runs-on: ${{ matrix.os }}

    steps:
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}

      - run: pytest tests/

```

Reusable Workflows

```

yaml
# .github/workflows/reusable-build.yml
on:
  workflow_call:           # Can be called by other workflows
  inputs:
    image-name:
      required: true
      type: string

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - run: docker build -t ${{ inputs.image-name }} .

```

```

yaml
# .github/workflows/main.yml
jobs:
  call-build:
    uses: ../.github/workflows/reusable-build.yml
    with:
      image-name: my-app:latest

```

Scheduled Workflows (cron)

```
yaml

on:
  schedule:
    - cron: "0 2 * * *"    # Run at 2am every day
```

10. Troubleshooting

Workflow not triggering?

```
yaml

# Check your branch name matches exactly
on:
  push:
    branches: [main]    # ← is your branch 'main' or 'master'?
```

Secret not found?

- Make sure the secret name matches exactly (case-sensitive)
- Secrets are not passed to workflows triggered by forks

Docker build fails in CI but works locally?

```
yaml

# Make sure .dockerignore isn't excluding files you need
# Add a debug step to see what files are available:
- name: List files
  run: ls -la
```

SSH deploy failing?

```
bash
```

```
# Test your SSH key locally first
ssh -i your-key.pem user@your-server

# Make sure the server's authorized_keys has your public key
cat ~/.ssh/authorized_keys
```

Viewing logs

In the **Actions** tab on GitHub:

- Click the workflow run
- Click any job to expand it
- Click any step to see its full output
- Failed steps show in red with the error message

Re-running a failed workflow

In the Actions tab → click the failed run → **Re-run jobs** → **Re-run failed jobs**

11. Cheat Sheet

Triggers

```
yaml

on:
  push:                                # On every push
  push:                                # Only these branches
    branches: [main, develop]
  pull_request:
    branches: [main]                 # PRs targeting main
  release:
    types: [published]               # On GitHub release
  schedule:
    - cron: "0 0 * * 1"              # Every Monday midnight
  workflow_dispatch:                  # Manual trigger button
```

Common Actions

```
yaml
```

```

uses: actions/checkout@v4           # Checkout your code
uses: actions/setup-python@v5       # Set up Python
uses: actions/setup-node@v4         # Set up Node.js
uses: docker/login-action@v3        # Log in to registry
uses: docker/build-push-action@v5   # Build and push image
uses: docker/metadata-action@v5     # Generate image tags
uses: docker/setup-buildx-action@v3 # Enable build cache
uses: appleboy/ssh-action@v1        # SSH into server

```

Context Variables

```

yaml

${{ github.sha }}           # Full commit SHA
${{ github.ref_name }}      # Branch or tag name
${{ github.repository }}    # owner/repo
${{ github.actor }}         # Who triggered the run
${{ github.event_name }}    # push, pull_request, etc.
${{ secrets.MY_SECRET }}    # Your stored secret
${{ env.MY_VAR }}           # Environment variable

```

Job Control

```

yaml

needs: [test, build]          # Run after these jobs complete
if: github.ref == 'refs/heads/main' # Conditional execution
continue-on-error: true      # Don't fail pipeline if this job fails
timeout-minutes: 10         # Fail if job takes longer than 10 min

```

Step Control

```

yaml

- name: Step name
  if: success()           # Only if previous steps passed
  if: failure()           # Only if a previous step failed
  if: always()            # Run regardless
  continue-on-error: true # Don't fail job if this step fails
  timeout-minutes: 5
  env:
    MY_VAR: value        # Step-level env var

```

Next Steps

- **Branch protection rules** — require CI to pass before merging PRs (Settings → Branches)
- **Dependabot** — auto-update GitHub Actions versions and Docker base images
- **CodeQL** — free security scanning built into GitHub Actions
- **Slack/Discord notifications** — post build results to your team chat
- **Kubernetes deployments** — replace `docker compose` with `kubect1` for production scale